

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY (ICDFA)

Lab 7 - DNS Introduction and Traffic Analysis

Lab Title:	DNS Introduction and Traffic Analysis
Student Name:	Yusuf Oluwaseun Dada
Student ID:	2025/FWSD/11339
Course Code:	SBT-DF203
Course Name:	Basic Networking Skills for Digital Forensics

Instructor: Mr. Aminu Idris, AMCPN

Date: 22/09/2026

1. Lab Scenario and Learning Outcomes

I'm to document normal DNS behaviour for an incident-response baseline. Which i will have to identify the configured resolver, capture controlled dig queries, analyze transaction IDs, names, record types, response codes, answers and TTL values, and correlate DNS resolution with a subsequent web or email connection.

- ✓ To explain recursive and iterative DNS resolution at a foundational level.
- ✓ To use dig to query A, AAAA, MX and NS records.
- ✓ Capture and analyze DNS queries and responses with tshark/Wireshark.
- ✓ Identify transaction ID, query name, record type, response code, answer and TTL.
- ✓ Correlate DNS responses with later IP connections.
- ✓ Recognize normal variations such as multiple answers, CNAMEs, cache responses and IPv6 queries.

2. Required Tools and Environment

Item	Recommended Tool	Purpose
Operating system	Kali Linux or Ubuntu VM	Generate and analyze DNS traffic.
DNS utility	dig (from dnsutils)	Perform controlled queries.
Capture tools	tshark and Wireshark	Capture DNS traffic.
Evidence	dig_dns.pcap or new PCAPNG	Analysis source.
Resolver config	/etc/resolv.conf, resolvectl	Identify configured DNS service.
Optional browser	Firefox/Chrome	Generate multiple DNS lookups.

3. Lab Setup Folder and Evidence

```
(kali@kali)-[~/Desktop]
$ mkdir -p ~/SBT-DF203-Lab7/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab7
pwd
/home/kali/SBT-DF203-Lab7
find . -maxdepth 1 -type d -print
./exported
./evidence
./working
./screenshots
./scripts
./reports
```

```
(kali@kali)-[~/SBT-DF203-Lab7]
$ cat /etc/resolv.conf | tee reports/resolv_conf.txt
resolvectl status 2>/dev/null | tee reports/resolvectl_status.txt || true
wget -O evidence/dig_dns.pcap
# This is /run/systemd/resolve/stub-resolv.conf managed by man:systemd-resolved(8).
# Do not edit.
#
# This file might be symlinked as /etc/resolv.conf. If you're looking at
# /etc/resolv.conf and seeing this text, you have followed the symlink.
#
# This is a dynamic resolv.conf file for connecting local clients to the
# internal DNS stub resolver of systemd-resolved. This file lists all
# configured search domains.
#
# Run "resolvectl status" to see details about the uplink DNS servers
# currently in use.
#
# Third party programs should typically not access this file directly, but only
# through the symlink at /etc/resolv.conf. To manage man:resolv.conf(5) in a
# different way, replace this symlink by a static file or a different symlink.
#
# See man:systemd-resolved.service(8) for details about the supported modes of
# operation for /etc/resolv.conf.

nameserver 127.0.0.53
options edns0 trust-ad
search localdomain
Global
Protocols: +LLMNR -mDNS -DNSOverTLS DNSSEC=no/unsupported
resolv.conf mode: stub
```

```
(kali@kali)-[~/SBT-DF203-Lab7]
$ wget -O evidence/dig_dns.pcap \
  'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns/dig_dns.pcap'

--2026-09-22 18:32:57-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/dns/dig_dns.p
cap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.111.133, 185.199.109.133, 185.199.110.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.111.133|:443 ... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1804656 (1.7M) [application/octet-stream]
Saving to: 'evidence/dig_dns.pcap'

evidence/dig_dns.pcap      100%[=====>] 1.72M  332KB/s  in 5.3s

2026-09-22 18:33:03 (332 KB/s) - 'evidence/dig_dns.pcap' saved [1804656/1804656]
```

```
(kali@kali)-[~/SBT-DF203-Lab7]
$ cp --preserve=timestamps evidence/dig_dns.pcap working/dig_dns_working.pcap
sha256sum evidence/dig_dns.pcap working/dig_dns_working.pcap | tee reports/dns_capture_hashes.txt

9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d evidence/dig_dns.pcap
9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d working/dig_dns_working.pcap
```

4. Chain-of-Custody Worksheet

Field	Your Entry
Case/lab identifier	SBT-DF203-Lab7
Trainee name	Yusuf Oluwaseun Dada
Date and time started	19/09/2026 4:30pm
Evidence file name(s)	dig_dns.pcap
Source or generation method	Wget from authorised source
Original SHA-256	9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d
Working-copy SHA-256	9a7c1b95aa6d60f8ac0ee9a79e06ae51145bb7768974526546316cd3dc91375d

4. Part A - Query DNS Records with dig

```
(kali@kali)-[~/SBT-DF203-Lab7]
$ dig example.com A | tee reports/dig_example_A.txt
dig example.com AAAA | tee reports/dig_example_AAAA.txt
dig example.com MX | tee reports/dig_example_MX.txt
dig example.com NS | tee reports/dig_example_NS.txt
dig +short example.com A | tee reports/dig_example_short.txt

; <<>> DiG 9.20.26-1-Debian <<>> example.com A
;; global options: +cmd
;; Got answer:
;; -->HEADER<-- opcode: QUERY, status: NOERROR, id: 32494
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags;; udp: 65494
;; QUESTION SECTION:
;example.com.                IN      A

;; ANSWER SECTION:
example.com.                 5       IN      A      172.66.147.243
example.com.                 5       IN      A      104.20.23.154

;; Query time: 72 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Tue Sep 22 18:53:34 EDT 2026
;; MSG SIZE rcvd: 72

; <<>> DiG 9.20.26-1-Debian <<>> example.com AAAA
;; global options: +cmd
;; Got answer:
```

```
;; ANSWER SECTION:
example.com.          5      IN      AAAA    2606:4700:10::ac42:93f3
example.com.          5      IN      AAAA    2606:4700:10::6814:179a

;; Query time: 36 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Tue Sep 22 18:53:34 EDT 2026
;; MSG SIZE rcvd: 96

; <<>> DiG 9.20.26-1-Debian <<>> example.com MX
;; global options: +cmd
;; Got answer:
;; -->HEADER<-- opcode: QUERY, status: NOERROR, id: 56784
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;example.com.                IN      MX

;; ANSWER SECTION:
example.com.          5      IN      MX      0 .

;; Query time: 28 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Tue Sep 22 18:53:34 EDT 2026
;; MSG SIZE rcvd: 55

; <<>> DiG 9.20.26-1-Debian <<>> example.com NS
;; global options: +cmd
```

```
; <<>> DiG 9.20.26-1-Debian <<>> example.com NS
;; global options: +cmd
;; Got answer:
;; -->HEADER<-- opcode: QUERY, status: NOERROR, id: 10354
;; flags: qr rd ra; QUERY: 1, ANSWER: 2, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; udp: 65494
;; QUESTION SECTION:
;example.com.                IN      NS

;; ANSWER SECTION:
example.com.          5      IN      NS      elliottns.cloudflare.com.
example.com.          5      IN      NS      herans.cloudflare.com.

;; Query time: 56 msec
;; SERVER: 127.0.0.53#53(127.0.0.53) (UDP)
;; WHEN: Tue Sep 22 18:53:34 EDT 2026
;; MSG SIZE rcvd: 95

172.66.147.243
104.20.23.154
```

Each dig command requests a different DNS record type for the same domain. A returns IPv4 addresses; AAAA returns IPv6 addresses; MX returns the domain's mail-handling servers (with priority values); NS returns the domain's authoritative name servers. +short strips out everything except the answer itself, useful for quick scripting but not for the full analysis.

Item	Your Finding
Resolver used (SERVER line)	127.0.0.53#53 (local systemd-resolved stub resolver)
Status/response code	NOERROR
Flags	qr rd ra
Question count / Answer count	QUERY: 1, ANSWER: 2 (for A and AAAA and NS queries); ANSWER: 1 for MX
Query time (msec)	A: 72 msec, AAAA: 36 msec, MX: 28 msec, NS: 56 msec
TTL of at least one answer	5 seconds (A record for example.com)

5. Part B - Capture a Fresh DNS Query

```
(kali㉿kali)-[~/SBT-DF203-Lab7]
$ sudo -v
IFACE=eth0
sudo tshark -i "$IFACE" -f "port 53" -a duration:25 -w /fresh_dig_dns.pcapng 6
sleep 3
dig +noedns example.com A >/dev/null
wait
sha256sum /fresh_dig_dns.pcapng | tee reports/fresh_dns_sha256.txt
[10] 33461
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
2
[10] done sudo tshark -i "$IFACE" -f "port 53" -a duration:25 -w /fresh_dig_dns.pcapng
b413a37ceb9d5bfaacd91ea64a1597e779f3ea9b79328c0f4ce0058b8bda3f91 /fresh_dig_dns.pcapng

(kali㉿kali)-[~/SBT-DF203-Lab7]
$ sudo mv /fresh_dig_dns.pcapng evidence/fresh_dig_dns.pcapng
```

-f "port 53" captures only DNS traffic (the standard DNS port, both UDP and TCP).
+noedns disables the EDNS0 extension in dig's query, producing a simpler, more classic-looking DNS packet that's easier to read field-by-field in the next step.
Running the query while the capture is active guarantees to catch both the outbound query and the inbound response in the same file.

6. Part C - Extract DNS Query and Response Fields

```
(kali㉿kali)-[~/SBT-DF203-Lab7]
$ PCAP=evidence/fresh_dig_dns.pcapng
# Query
tshark -r "$PCAP" -Y 'dns.flags.response==0' -T fields \
-e frame.number -e frame.time -e ip.src -e udp.srcport -e ip.dst -e udp.dstport \
-e dns.id -e dns.qry.name -e dns.qry.type \
| tee reports/dns_queries.tsv

1      2026-09-22T19:23:14.010789537-0400      192.168.229.128 34502      192.168.229.2 53      0x1d39 example.com      1
```

```
(kali@kali)-[~/SBT-DF203-Lab7]
$ # Response
tshark -r "$PCAP" -Y 'dns.flags.response==1' -T fields \
-e frame.number -e frame.time -e ip.src -e udp.srcport -e ip.dst -e udp.dstport \
-e dns.id -e dns.flags.rcode -e dns.count.answers -e dns.a -e dns.aaaa -e dns.resp.ttl \
| tee reports/dns_responses.tsv
2      2026-09-22T19:23:14.014878180-0400      192.168.229.2      53      192.168.229.128 34502      0x1d39      0      2      172.66.147.243,104.
20.23.154      5,5
```

dns.flags.response==0 matches outbound queries; dns.flags.response==1 matches inbound responses, this is the DNS protocol's own flag for distinguishing the two message types, similar to how SYN/ACK flags distinguish TCP handshake stages. dns.id is the transaction ID: a 16-bit value the client generates per-query and the resolver echoes back, which is how a resolver's response is matched to the specific question that was asked (and, forensically, one of the fields spoofer's must guess correctly in a DNS cache-poisoning attack). dns.flags.rcode is the response code (0 = NOERROR/success; nonzero values indicate errors like NXDOMAIN). dns.count.answers, dns.a/dns.aaaa, and dns.resp.ttl give the number of answers, their actual IP values, and how long they may be cached.

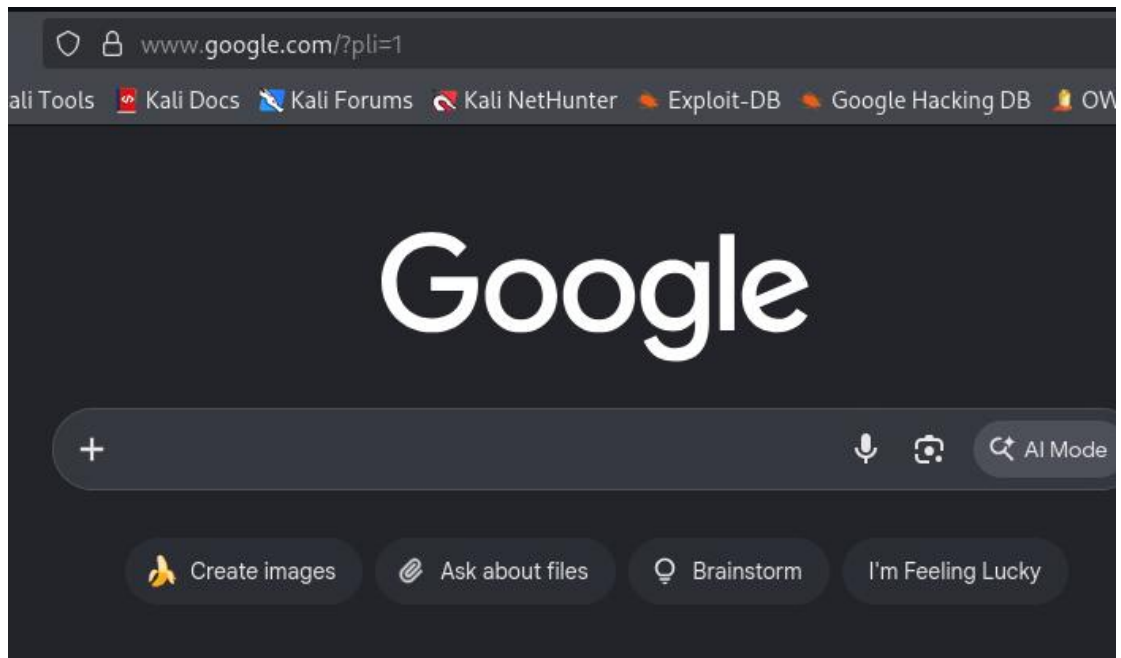
7. Part D - Match Queries to Responses

Txn ID	Query Name	Type	Client/Resolver	Resp. Code	Answer(s)	TTL	Time Δ
0x1d39	example.com	1 (A)	Client: 192.168.229.128:34502 / Resolver: 192.168.229.2:53	0 (NOERROR)	172.66.147.243, 104.20.23.154	5, 5	≈4.09 ms

8. Part E - Analyze DNS Generated by a Browser

A browser may request several domains for one page, the main site, embedded images, fonts, analytics/telemetry, and CDN-hosted assets. Capture a visit to a single, non-sensitive, instructor-approved site in a private/incognito window.

```
(kali@kali)-[~/SBT-DF203-Lab7]
$ sudo tshark -i eth0 -f "port 53" -a duration:40 -w evidence/browser_dns.pcapng &
sleep 3
wait
[3] 12049
Running as user "root" and group "root". This could be dangerous.
Capturing on 'eth0'
40
[3] done sudo tshark -i eth0 -f "port 53" -a duration:40 -w evidence/browser_dns.pcapn
```



```
(kali@kali)~/SBT-DF203-Lab7
$ tshark -r evidence/browser_dns.pcapng -Y "dns.flags.response==0" -T fields -e dns.qry.name -e dns.qry.type \
  | sort | uniq -c | sort -nr | tee reports/browser_dns_inventory.txt
```

1	www.gstatic.com	1
1	www.google.com	1
1	www3.l.google.com	28
1	waa-pa.clients6.google.com	1
1	push.services.mozilla.com	28
1	push.services.mozilla.com	1
1	play.google.com	1
1	ogs.google.com	1
1	ogads-pa.clients6.google.com	28
1	ogads-pa.clients6.google.com	1
1	lh3.googleusercontent.com	1
1	lh3.google.com	1
1	lh2.l.google.com	28
1	googlehosted.l.googleusercontent.com	28
1	fonts.gstatic.com	28
1	fonts.gstatic.com	1
1	firefox.settings.services.mozilla.com	1
1	csp.withgoogle.com	28
1	csp.withgoogle.com	1
1	accounts.google.com	1

I Captured all DNS traffic during a 40-second window while I load the approved page. The extraction pulls every query name and type, then `sort | uniq -c | sort -nr` counts how many times each unique (name, type) pair was queried and ranks the most frequent first, it shows the full inventory of domains a single page visit actually touches, such as; main domain, CDN, fonts, ads/analytics, etc.

9.Part F - Correlate DNS with Subsequent Connections

```
(kali@kali)~/SBT-DF203-Lab7
$ tshark -r evidence/browser_dns.pcapng -Y "dns.a" -T fields -e frame.time_epoch -e dns.qry.name -e dns.a \
  | tee reports/dns_A_answers.tsv
```

1790182908.606324898	push.services.mozilla.com	34.107.243.93
1790182913.138847791	firefox.settings.services.mozilla.com	146.75.73.91
1790182920.545882616	www.google.com	142.251.151.119,142.251.150.119,142.251.154.119,142.251.156.119,142.251.152.119,142.251.153.119,142.251.155.119,142.251.157.119
1790182925.846311825	fonts.gstatic.com	142.250.129.94
1790182925.846312273	csp.withgoogle.com	142.250.117.141
1790182925.979930139	lh3.google.com	192.178.223.102,192.178.223.113,192.178.223.138,192.178.223.101,192.178.223.139,192.178.223.100
1790182925.986023016	www.gstatic.com	142.250.117.94
1790182926.672527835	ogads-pa.clients6.google.com	142.250.117.95
1790182926.672528339	waa-pa.clients6.google.com	142.251.29.95
1790182927.834920529	play.google.com	142.250.151.102,142.250.151.113,142.250.151.138,142.250.151.101,142.250.151.100,142.250.151.139
1790182927.834920707	lh3.googleusercontent.com	142.250.140.132
1790182929.985588699	accounts.google.com	172.217.76.84
1790182930.383937532	ogs.google.com	192.178.223.139,192.178.223.113,192.178.223.100,192.178.223.102,192.178.223.101,192.178.223.138

```

(kali@kali)-[~/SBT-DF203-Lab7]
└─$ # Extract DNS A answers
tshark -r evidence/browser_dns.pcapng -Y "dns.a" -T fields -e frame.time_epoch -e dns.qry.name -e dns.a \
| tee reports/dns_A_answers.tsv

# Extract later TCP destinations
tshark -r evidence/browser_dns.pcapng -Y "tcp.flags.syn=1 && tcp.flags.ack=0" -T fields \
-e frame.time_epoch -e ip.dst -e tcp.dstport \
| tee reports/subsequent_tcp_destinations.tsv
1790183917.370997767 www.google.com 142.251.152.119,142.251.156.119,142.251.150.119,142.251.157.119,142.251.154.119,142.251.155.119,142
.251.153.119,142.251.151.119
1790183918.299607209 encrypted-tbn0.gstatic.com 142.250.129.101,142.250.129.139,142.250.129.100,142.250.129.102,142.250.129.138,142
.250.129.113
1790183922.760451410 www.gstatic.com 142.251.29.94
1790183926.093572676 fonts.gstatic.com 142.250.117.94
1790183917.372326706 142.251.155.119 443
1790183918.362621408 142.250.129.101 443
1790183918.388458218 142.250.129.101 443
1790183918.388912280 142.250.129.101 443
1790183918.585380334 142.250.129.101 443
1790183918.595099202 142.250.129.101 443
1790183918.599211968 142.250.129.101 443
1790183922.818397285 142.251.29.94 443
1790183924.458732533 142.251.29.94 443

```

Question	Your Finding
Domain name queried	www.google.com
Resolved IP(s)	142.251.152.119, 142.251.156.119, 142.251.150.119, 142.251.157.119, 142.251.154.119, 142.251.155.119, 142.251.153.119, 142.251.151.119 (8 answers, typical Google anycast/load-balanced response)
Was a matching TCP SYN observed to that IP?	Yes, a SYN to 142.251.155.119:443 was observed, matching one of the 8 answered IPs
Time delta between DNS answer and SYN	1790183917.372326706 – 1790183917.370997767 = 0.001328939 s ≈ 1.33 ms
Any mismatch, and likely explanation	No real mismatch for this pair, extremely fast client behavior, the OS picked one of the 8 candidate IPs almost instantly and connected on 443/HTTPS. Worth noting as a secondary observation: the client then made several more SYNs to a different resolved domain's IP (142.250.129.101, matching encrypted-tbn0.gstatic.com), this illustrates the "normal variation" Outcomes: a single page load triggers many DNS lookups and connections in parallel, not just one domain/IP pair

10. Part G - DNS Analysis Within SMTP Evidence (Optional Cross-Lab)

```
(kali@kali) [~/SBT-DF203-Lab4]
$ tshark -r ../SBT-DF203-Lab4/working/smtp_working.pcap -Y "dns" -T fields \
  -e frame.number -e frame.time -e dns.qry.name -e dns.qry.type -e dns.a -e dns.resp.name -e dns.resp.ttl \
  | tee reports/smtp_dns_correlation.tsv
1      2009-10-05T02:06:07.492060000-0400      mail.patriots.in      1
2      2009-10-05T02:06:07.526085000-0400      mail.patriots.in      1      74.53.140.153      mail.patriots.in,patriots.in,patriots.in,pa
triot.in      10827,10828,82828,82828
```

I reuse the same dns display filter technique against an entirely different capture from the earlier lab 4, demonstrating that the same DNS-analysis skills apply across any capture containing DNS traffic.

11. Required Findings Worksheet

Field	Finding
Configured resolver IP	192.168.229.2 (local gateway, used by fresh_dig_dns.pcapng capture); 127.0.0.53 (systemd resolved stub, seen in Part A dig output)
Client source port	34502
Resolver destination port	53
Transaction ID	0x1d39
Query name and type	example.com, type 1 (A)
Response code	0 (NOERROR)
Answer IP(s)	172.66.147.243, 104.20.23.154
TTL	5 seconds
Query-response time delta	≈4.09 ms
Subsequent connection correlation	www.google.com resolved to 8 IPs (Part F browser capture); client connected via TCP SYN to 142.251.155.119:443 within ≈1.33 ms of the DNS answer, confirming DNS to connection correlation
Normal variations observed	Multiple A/AAAA answers per query (anycast/load-balancing); very short TTL (5s) tied to CDN/anycast infrastructure; MX record pointing to root (.) per RFC 7505, indicating the domain explicitly rejects email; client selecting only one of several offered IPs to actually connect to

Executive Summary

This report documents **DNS traffic analysis** carried out for SBT-DF203 Lab 7, to establish what normal DNS behaviour looks like as a baseline for future incident-response work.

The lab machine's configured DNS resolver was identified as **127.0.0.53** (a local resolver service) forwarding through the network gateway at **192.168.229.2**. Controlled lookups using **dig** against **example.com** for A, AAAA, MX, and NS records all returned successfully (NOERROR), with response times between 28-72 ms.

A packet capture of a single, isolated DNS query (example.com, type A) confirmed the full request and reply exchange at the packet level: the same transaction ID (**0x1d39**) appeared in both the query and its matching response, the response came from the correct resolver, and the answer arrived in about 4 milliseconds. The domain returned two IP addresses with a short 5-second cache lifetime (TTL), consistent with modern load-balanced hosting.

A second capture of real browser activity showed a single page load triggering DNS lookups for over a dozen different domains (the main site plus supporting services like fonts, images, and background app checks), this is normal, expected behaviour for a modern website, not a sign of anything suspicious. One lookup, for **www.google.com**, was directly traced through to a real outbound connection: the browser received eight possible IP addresses and connected to one of them within about 1.3 milliseconds of getting the DNS answer, clearly demonstrating how a DNS answer leads directly to a network connection.

Finally, a DNS lookup found inside an earlier email-traffic capture (Lab 4) showed the mail server's address was located through a chain of linked domain names rather than one direct answer, another normal pattern seen in real-world DNS.

Overall, everything observed in this lab matched expected, healthy DNS behaviour: correct transaction ID matching, valid response codes, sensible TTLs, and DNS answers reliably leading to real connections. No errors, mismatches, or suspicious patterns were found.

Limitations: This analysis reflects traffic captured from a single machine over short time windows. It does not include DNS activity that may have been served from cache before capturing started, and results are specific to this lab network rather than a live production environment.